

Python desde Cero.

Clase 1: Introducción y Conceptos Básicos

En esta primera clase aprenderemos:

Historia de Python, sus ventajas y desventajas, aplicaciones principales, razones de su popularidad y conceptos básicos de programación.

¿A quién está dirigido?

Este curso está especialmente diseñado para principiantes sin conocimientos previos en programación. Comenzaremos desde los conceptos más básicos.

Metodología

Aprenderemos de forma práctica y progresiva, con ejemplos claros y ejercicios que te permitirán dominar uno de los lenguajes de programación más populares del mundo.

Historia de Python

1989

Creación: Guido van Rossum comienza a desarrollar Python durante las vacaciones de Navidad mientras trabajaba en el CWI (Centrum Wiskunde & Informatica) en los Países Bajos.

1991

Primera versión: Se lanza Python 0.9.0, la primera versión pública del lenguaje. Ya incluía características como funciones, módulos, manejo de excepciones y tipos de datos como listas y diccionarios.

1994

Python 1.0: Se publica la primera versión estable con mejoras significativas como funciones lambda, map, filter y reduce.

2000

Python 2.0: Introducción de list comprehensions y un recolector de basura para eliminar referencias circulares. Comienza la era de Python 2.x.

2008

Python 3.0: Una revisión importante del lenguaje que no es compatible con versiones anteriores. Mejora el soporte para Unicode y corrige inconsistencias del diseño.

"Python es un lenguaje que destaca por su simplicidad y legibilidad. Mi objetivo siempre fue crear un lenguaje que hiciera la programación divertida."

- Guido van Rossum

💡 ¿Sabías que...?

El nombre "Python" no proviene de la serpiente, sino del grupo humorístico británico "Monty Python's Flying Circus". Guido van Rossum era fan del programa y decidió darle este nombre a su creación.

Evolución Reciente de Python

Python 3.6

Diciembre 2016

- ✓ Introducción de f-strings para formateo de cadenas
- ✓ Variable annotations
- ✓ Mejoras en el módulo asyncio

Python 3.7

Junio 2018

- ✓ Mejoras significativas en rendimiento
- ✓ Data classes
- ✓ Postponed evaluation of annotations

Python 3.8

Octubre 2019

- ✓ Operador walrus (:=) para asignaciones
- ✓ Posicionamiento de parámetros
- ✓ f-strings soportan =

Python 3.9

Octubre 2020

- ✓ Operadores de unión y actualización de diccionarios
- ✓ Mejoras en tipos genéricos
- ✓ Parser optimizado

Python 3.10

Octubre 2021

- ✓ Pattern matching (similar a switch)
- ✓ Mejores mensajes de error
- ✓ Union types en parámetros

Python 3.11+

Octubre 2022 - Presente

- ✓ Mejoras de velocidad (20-60% más rápido)
- ✓ Mejor gestión de excepciones
- ✓ Mejoras en tipado (typing)

Python en la actualidad

Python sigue siendo mantenido por la **Python Software Foundation (PSF)**, una organización sin fines de lucro fundada en 2001.

Hoy en día, Python es uno de los lenguajes de programación más populares del mundo, utilizado ampliamente en áreas como desarrollo web, ciencia de datos, inteligencia artificial, automatización, DevOps y más.

Ventajas de Python



Sintaxis clara y legible

La sintaxis de Python es simple, intuitiva y fácil de entender, lo que acelera el desarrollo y facilita la colaboración entre equipos. Su estilo de codificación promueve la escritura de código limpio y comprensible.



Amplia biblioteca estándar

Python incluye una extensa biblioteca estándar que cubre diversas áreas, desde manipulación de archivos hasta desarrollo web. Esto permite aprovechar herramientas existentes sin tener que escribir todo desde cero.



Desarrollo rápido de prototipos

La simplicidad y concisión de Python lo convierten en la herramienta ideal para el desarrollo rápido de prototipos. Permite probar ideas antes de comprometerse con implementaciones más extensas.



Gran comunidad y soporte

Python cuenta con una comunidad activa y extensa de desarrolladores que contribuyen con bibliotecas, tutoriales y resuelven dudas en foros. Esto proporciona un valioso respaldo y facilita el aprendizaje.



Versatilidad y portabilidad

Python funciona en diferentes sistemas operativos (Windows, macOS, Linux) sin modificaciones significativas. Su código puede ejecutarse en cualquier plataforma que tenga el intérprete de Python instalado.



Integración con otros lenguajes

Python puede integrarse fácilmente con otros lenguajes como C, C++ y Java. Esto permite aprovechar código existente y obtener un rendimiento óptimo cuando sea necesario.

"Python es un lenguaje que enfatiza la legibilidad y la productividad del programador. Su filosofía se centra en la simplicidad y la elegancia, permitiendo expresar conceptos en menos líneas de código que lenguajes como C++ o Java."

Desventajas de Python



Velocidad de ejecución

Al ser un lenguaje interpretado, Python puede ser más lento en comparación con lenguajes compilados como C++ o Java. Esto puede ser una limitación en aplicaciones que requieren rendimiento extremadamente rápido.



No ideal para desarrollo móvil

Aunque existen frameworks como Kivy o BeeWare, Python no es considerado el lenguaje principal para el desarrollo móvil. Otros lenguajes como Swift o Kotlin son preferidos en este contexto.



Interpretación y ejecución

El hecho de que Python sea interpretado puede ser una desventaja en términos de velocidad de ejecución. Sin embargo, esta desventaja se ve mitigada por las implementaciones Just-In-Time (JIT) como PyPy.



Gestión de memoria automática

Aunque es una ventaja para muchos, puede ser una desventaja en aplicaciones que requieren un control preciso de los recursos de memoria, ya que Python no proporciona la misma flexibilidad que lenguajes de bajo nivel.



Problemas de comprensión asíncrona

Aunque Python ha mejorado en la gestión de operaciones asíncronas, algunos desarrolladores encuentran desafíos al trabajar con código asíncrono, especialmente en comparación con lenguajes diseñados específicamente para tareas concurrentes.



Problemas de integración con C/C++

Aunque Python se puede integrar con C y C++, puede haber desafíos y complejidades asociadas con la interacción entre estos lenguajes, especialmente en proyectos más grandes.

A pesar de estas limitaciones, Python sigue siendo una excelente opción para la mayoría de los proyectos de desarrollo. La clave está en evaluar si estas desventajas son críticas para tu caso de uso específico.

Principales Usos de Python



Ciencia de Datos y Análisis

Python se ha convertido en el lenguaje preferido para análisis de datos, estadísticas y visualización gracias a sus potentes bibliotecas.

Bibliotecas: Pandas, NumPy, Matplotlib



Inteligencia Artificial

La simplicidad de Python y su potente ecosistema lo convierten en la opción preferida para desarrollar soluciones de IA y aprendizaje automático.

Bibliotecas: TensorFlow, PyTorch, Keras



Desarrollo Web

Python permite desarrollar sitios web robustos y escalables con menos código, haciéndolos más ligeros y optimizados.

Frameworks: Django, Flask, FastAPI



Automatización y DevOps

Python es excelente para automatizar tareas repetitivas, administrar servidores y configurar infraestructuras.

Ejemplos: Scripts de automatización, Ansible, Jenkins



Desarrollo de Juegos

Python se utiliza para crear prototipos de juegos y también para desarrollar juegos completos gracias a bibliotecas especializadas.

Bibliotecas: Pygame, Panda3D, Arcade



Blockchain y Criptomonedas

Python es utilizado para desarrollar aplicaciones blockchain y criptomonedas debido a su versatilidad, seguridad y velocidad.

Ejemplos: Smart contracts, aplicaciones descentralizadas

★ Empresas que usan Python extensivamente

Python es utilizado por grandes empresas tecnológicas como Google, Facebook, Instagram, Spotify, Netflix, Dropbox y muchas más. Google adoptó Python desde sus primeros días, y hoy es uno de los tres lenguajes oficiales de la compañía junto con Java y C++.

¿Por qué Python es tan popular hoy en día?

1 Curva de aprendizaje accesible

Python fue diseñado con la legibilidad como prioridad. Su sintaxis clara y similar al inglés lo hace ideal para principiantes. Puedes comenzar a programar de manera efectiva en poco tiempo, un factor clave para estudiantes y profesionales que buscan adquirir nuevas habilidades.

2 Auge de la ciencia de datos e IA

El crecimiento explosivo de la ciencia de datos, el aprendizaje automático y la inteligencia artificial ha impulsado enormemente la popularidad de Python. Bibliotecas como Pandas, NumPy, TensorFlow y PyTorch han convertido a Python en el lenguaje preferido para estas disciplinas.

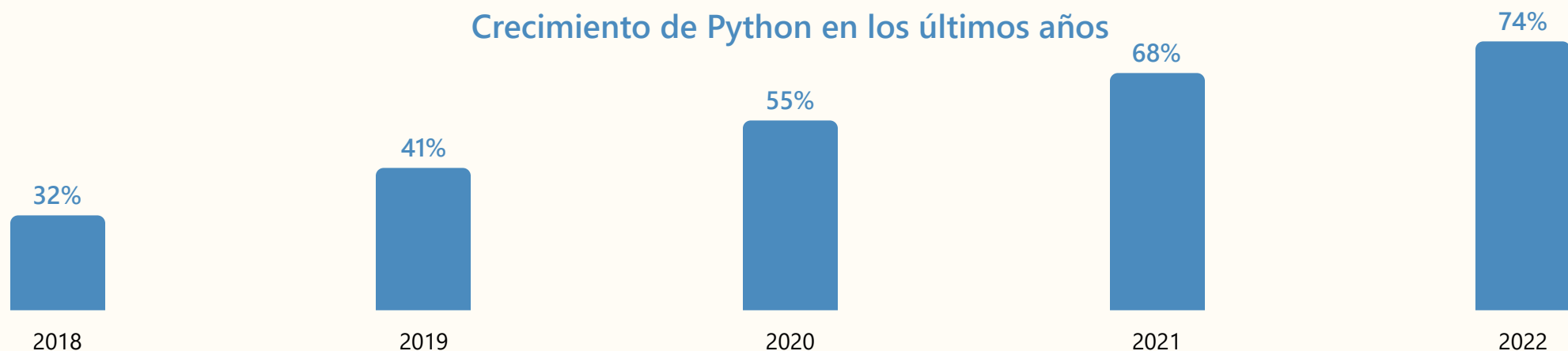
3 Ecosistema robusto

Con más de 350,000 paquetes disponibles en PyPI (Python Package Index), existe una biblioteca para prácticamente cualquier tarea. Esta riqueza de recursos permite a los desarrolladores construir soluciones complejas sin reinventar la rueda.

4 Adopción corporativa

Grandes empresas como Google, Facebook, Netflix y Amazon utilizan Python ampliamente en sus infraestructuras. Esta adopción corporativa ha creado una gran demanda de desarrolladores de Python en el mercado laboral, impulsando aún más su popularidad.

Crecimiento de Python en los últimos años



Python comparado con otros lenguajes

Lenguaje	Tipo	Fortalezas	Casos de uso
 Python	Interpretado, alto nivel	Fácil de aprender , sintaxis clara, desarrollo rápido	Ciencia de datos, IA, desarrollo web, automatización, prototipado
 JavaScript	Interpretado, alto nivel	Frontend y backend, asíncrono, ubicuo en navegadores	Aplicaciones web, desarrollo frontend, Node.js para backend
 Java	Compilado, alto nivel	Portabilidad, rendimiento, multihilo robusto	Aplicaciones empresariales, Android, sistemas financieros
 C++	Compilado, bajo/medio nivel	Alto rendimiento, control de memoria, eficiencia	Videojuegos, aplicaciones de alto rendimiento, sistemas operativos

💡 ¿Qué hace a Python diferente?

Python se destaca por su **filosofía de diseño** que prioriza la legibilidad y la simplicidad. A diferencia de otros lenguajes, Python enfatiza:

- La legibilidad del código como valor principal ("Python es pseudocódigo ejecutable")
- El enfoque de "Una forma obvia de hacerlo" en lugar de ofrecer múltiples alternativas
- El uso de indentación para definir bloques de código (en lugar de llaves o palabras clave)
- Un ecosistema científico y de datos inigualable con fuerte soporte de la comunidad

Comenzando con Python: El Zen de Python

¿Qué es el Zen de Python?

El Zen de Python es una colección de 19 principios que guían el diseño del lenguaje Python. Fue escrito por Tim Peters, un desarrollador importante de Python, y encapsula la filosofía del lenguaje.

```
# Para ver el Zen de Python, sólo necesitas ejecutar:  
>>> import this
```

Algunos principios del Zen de Python

"Hermoso es mejor que feo."

"Explícito es mejor que implícito."

"Simple es mejor que complejo."

"Complejo es mejor que complicado."

"La legibilidad cuenta."

"Los casos especiales no son tan especiales como para romper las reglas."

Estos principios no son solo reglas de sintaxis, sino una filosofía que guía cómo deberíamos escribir y pensar sobre el código. Python fue diseñado para ser legible y expresivo, y estos principios reflejan esa intención.

💡 La importancia de PEP 8

PEP 8 es la guía de estilo oficial para escribir código Python. Fue creada para garantizar la coherencia y legibilidad en el código Python. Seguir PEP 8 hace que tu código sea más fácil de leer y mantener por ti mismo y por otros programadores.

Algunos de sus principios incluyen:

- Usar 4 espacios para la indentación
- Límite de 79 caracteres por línea
- Convenciones para nombrar variables, funciones y clases
- Consejos sobre el uso de espacios en blanco

Variables y Tipos de Datos Básicos

📦 ¿Qué es una variable?

Una variable en Python es como un contenedor que almacena datos. Piensa en ella como una caja con una etiqueta (nombre) que contiene un valor.

```
# Asignación de variables
nombre = "Ana"
edad = 25
precio = 19.99
```

💡 Características de las variables

- No necesitan ser declaradas con un tipo específico
- El tipo se determina automáticamente en tiempo de ejecución
- Se pueden reasignar a diferentes valores y tipos
- Son sensibles a mayúsculas y minúsculas (edad ≠ Edad)

Tipos de datos básicos en Python

Números

Enteros (int), decimales (float) y complejos (complex)

```
entero = 42
decimal = 3.14159
complejo = 3+2j
```

Texto

Cadenas de caracteres (str)

```
nombre = "María"
frase = 'Hola, mundo!'
parrafo = """Texto de
múltiples líneas"""
```

Booleanos

Valores lógicos verdadero/falso (bool)

```
es_estudiante = True
tiene_auto = False
```

Valor Nulo

Representa la ausencia de valor

```
valor_vacio = None
```

💡 Verificar el tipo de datos

Para conocer el tipo de una variable, puedes usar la función `type()`:

```
edad = 25
print(type(edad)) # Salida: <class 'int'>

nombre = "Juan"
print(type(nombre)) # Salida: <class 'str'>
```

Tipos de Datos de Colección

☰ Listas (list)

Secuencias ordenadas y modificables de elementos. Pueden contener elementos de diferentes tipos.

```
# Creación de listas
colores = ["rojo", "verde", "azul"]
mixta = [1, "hola", 3.14, True]
```

- ✔ Mutables (se pueden modificar)
- ✔ Acceso por índice: `colores[0] = "rojo"`
- ✔ Admite métodos como `append()`, `insert()`, `remove()`

🔒 Tuplas (tuple)

Secuencias ordenadas e inmutables. Similar a las listas, pero no se pueden modificar una vez creadas.

```
# Creación de tuplas
coordenadas = (10, 20)
persona = ("Juan", 30, "Madrid")
```

- ✔ Inmutables (no se pueden modificar)
- ✔ Más rápidas que las listas
- ✔ Útiles para datos que no deben cambiar

📖 Diccionarios (dict)

Colecciones de pares clave-valor. Cada valor es accesible mediante su clave única.

```
# Creación de diccionarios
persona = {
    "nombre": "Ana",
    "edad": 25,
    "ciudad": "Barcelona"
}
```

- ✔ Mutables y no ordenados (en versiones < 3.7)
- ✔ Acceso por clave: `persona["nombre"] = "Ana"`
- ✔ Ideal para representar estructuras de datos complejas

👥 Conjuntos (set)

Colecciones no ordenadas de elementos únicos. Útiles para eliminar duplicados y operaciones matemáticas de conjuntos.

```
# Creación de conjuntos
frutas = {"manzana", "naranja", "plátano"}
numeros = set([1, 2, 3, 1, 2]) # {1, 2, 3}
```

- ✔ Elementos únicos (no admite duplicados)
- ✔ Soporta operaciones como unión, intersección, diferencia
- ✔ No permite acceso por índice

💡 ¿Cuál usar?

- **Lista:** Cuando necesites una colección ordenada que pueda cambiar
- **Tupla:** Cuando necesites una colección ordenada que NO cambie
- **Diccionario:** Cuando necesites pares clave-valor para búsquedas rápidas
- **Conjunto:** Cuando necesites elementos únicos o realizar operaciones de conjuntos

Operadores en Python

Operadores Aritméticos

+

Suma

```
x = 5 + 3
print(x) # Resultado: 8
```

-

Resta

```
x = 10 - 4
print(x) # Resultado: 6
```

*

Multiplicación

```
x = 3 * 4
print(x) # Resultado: 12
```

/

División

```
x = 10 / 2
print(x) # Resultado: 5.0
```

%

Módulo (resto)

```
x = 10 % 3
print(x) # Resultado: 1
```

**

Exponente

```
x = 2 ** 3
print(x) # Resultado: 8
```

Operadores de Comparación

==

Igual a

```
x = (5 == 5)
print(x) # Resultado: True
```

!=

Distinto a

```
x = (5 != 3)
print(x) # Resultado: True
```

<, >

Menor que, Mayor que

```
x = (3 < 5)
print(x) # Resultado: True
```

<=, >=

Menor o igual, Mayor o igual

```
x = (5 >= 5)
print(x) # Resultado: True
```

Operaciones con diferentes tipos de datos

Python permite combinar diferentes tipos de datos en operaciones:

```
# Concatenar cadenas
nombre = "Juan" + " Pérez" # "Juan Pérez"
```

```
# Repetir cadenas
puntos = "." * 3 # "..."
```

```
# Operaciones con booleanos
resultado = True + True # 2 (True se convierte a 1)
```



Estructuras de Control de Flujo

🔗 Condicionales: if-elif-else

Las estructuras condicionales permiten ejecutar diferentes bloques de código dependiendo de si una condición es verdadera o falsa.

```
edad = 18

if edad < 18:
    print("Eres menor de edad")
elif edad == 18:
    print("Acabas de cumplir 18")
else:
    print("Eres mayor de edad")
```

Nota: En Python, la indentación determina el bloque de código, no se usan llaves como en otros lenguajes.

🔄 Bucles: for

El bucle for se utiliza para iterar sobre una secuencia (como una lista, tupla, diccionario, conjunto o cadena).

```
# Iterando sobre una lista
frutas = ["manzana", "banana", "cereza"]
for fruta in frutas:
    print(fruta)

# Iterando sobre un rango
for i in range(5):
    print(i) # Imprime 0, 1, 2, 3, 4
```

🔄 Bucles: while

El bucle while ejecuta un bloque de código mientras una condición sea verdadera.

```
contador = 0

while contador < 5:
    print(contador)
    contador += 1

# También se puede usar break y continue
while True:
    respuesta = input("¿Salir? (s/n): ")
    if respuesta == "s":
        break
```

⚡ Control adicional: break, continue, pass

Python ofrece palabras clave adicionales para controlar el flujo dentro de los bucles.

- **break:** Sale inmediatamente del bucle
- **continue:** Salta a la siguiente iteración
- **pass:** No hace nada, actúa como un marcador

```
for i in range(10):
    if i == 3:
        continue # Salta el 3
    if i == 8:
        break # Termina el bucle en 8
    print(i)
```

💡 Comprensiones de listas

Python ofrece una sintaxis concisa llamada **comprensión de lista** para crear listas basadas en listas existentes:

```
# Forma tradicional
cuadrados = []
for x in range(10):
    cuadrados.append(x**2)

# Usando comprensión de lista
cuadrados = [x**2 for x in range(10)]

# Con condición
pares = [x for x in range(10) if x % 2 == 0]
```



Funciones en Python

¿Qué son las funciones?

Las funciones son bloques de código reutilizables diseñados para realizar una tarea específica. Ayudan a organizar el código, evitan la repetición y facilitan el mantenimiento.

```
# Definición básica de una función
def saludar():
    print("¡Hola, mundo!")

# Llamada a la función
saludar() # Imprime: ¡Hola, mundo!
```

Parámetros y Argumentos

Las funciones pueden aceptar datos de entrada (parámetros) y utilizar esos valores en su ejecución.

```
# Función con parámetros
def saludar_persona(nombre):
    print(f"¡Hola, {nombre}!")

# Llamada con argumento
saludar_persona("Ana") # Imprime: ¡Hola, Ana!
```

Retorno de valores

Las funciones pueden devolver valores que se pueden utilizar más adelante en el programa.

```
def sumar(a, b):
    return a + b

resultado = sumar(5, 3)
print(resultado) # Imprime: 8
```

Parámetros por defecto

Puedes asignar valores predeterminados a los parámetros, que se utilizarán si no se proporciona un argumento.

```
def saludar(nombre, saludo="Hola"):
    print(f"{saludo}, {nombre}!")

saludar("Juan") # Imprime: Hola, Juan!
saludar("María", "Buenos días") # Imprime: Buenos días, María!
```

Argumentos con nombre

Puedes especificar los argumentos por su nombre, lo que te permite cambiar el orden o omitir argumentos que tienen valores predeterminados:

```
def describir_persona(nombre, edad, profesion="estudiante"):
    print(f"{nombre} tiene {edad} años y es {profesion}.")

# Usando argumentos con nombre
describir_persona(edad=25, nombre="Carlos")
describir_persona(nombre="Laura", profesion="ingeniera", edad=30)
```

Buenas prácticas

- Usa nombres descriptivos para tus funciones
- Cada función debe realizar una sola tarea específica
- Incluye docstrings para documentar qué hace la función
- Verifica los valores de entrada cuando sea necesario
- Evita efectos secundarios inesperados

Módulos y Paquetes en Python

¿Qué son los módulos?

Un módulo es un archivo Python con extensión `.py` que contiene definiciones y declaraciones. Te permite organizar código relacionado en archivos separados y reutilizarlo.

```
# Archivo: matematicas.py
def sumar(a, b):
    return a + b

def restar(a, b):
    return a - b

PI = 3.14159
```

Importando módulos

Hay varias formas de importar módulos en Python para utilizar su funcionalidad en tu programa.

```
# Importar todo el módulo
import matematicas
resultado = matematicas.sumar(5, 3)

# Importar elementos específicos
from matematicas import sumar, PI
resultado = sumar(5, 3)

# Importar todo con alias
import matematicas as mat
resultado = mat.sumar(5, 3)
```

Paquetes en Python

Un paquete es una carpeta que contiene múltiples módulos (archivos `.py`) y un archivo especial `__init__.py`. Los paquetes permiten organizar el código en una estructura jerárquica.

```
mi_paquete/
├── __init__.py
├── modulo1.py
├── modulo2.py
└── subpaquete/
    ├── __init__.py
    └── modulo3.py

# Importar desde un paquete
from mi_paquete import modulo1
from mi_paquete.subpaquete import modulo3
```

Biblioteca estándar

Python incluye una rica biblioteca estándar con módulos predefinidos para tareas comunes, lo que se conoce como "baterías incluidas".

```
# Algunos módulos de la biblioteca estándar
import math # Funciones matemáticas
import random # Generación de números aleatorios
import datetime # Manipulación de fechas
import os # Interacción con el sistema operativo
import json # Manejo de datos JSON
```

Bibliotecas populares de Python

pandas

NumPy

Matplotlib

requests

TensorFlow

PyTorch

Django

Flask

FastAPI

Scikit-learn

Consejos para trabajar con módulos

- Utiliza entornos virtuales (`venv`) para gestionar paquetes y dependencias
- Instala paquetes externos con `pip install nombre_paquete`
- Mantén un archivo `requirements.txt` con las dependencias de tu proyecto
- Evita usar `from módulo import *` ya que puede causar conflictos de nombres



Entrada y Salida en Python

Mostrar información

La función `print()` es la forma más común de mostrar información en la consola.

```
# Imprimir texto simple
print("Hola mundo")

# Imprimir múltiples valores
print("Nombre:", "Ana", "Edad:", 25)

# Cambiar el separador
print("Python", "es", "genial", sep="-") # Python-es-genial
```

Recibir entrada del usuario

La función `input()` permite obtener datos ingresados por el usuario desde la consola.

```
# Entrada básica
nombre = input("Ingresa tu nombre: ")
print(f"Hola, {nombre}!")

# Convertir la entrada a número
edad = int(input("¿Cuál es tu edad? "))
print(f"El próximo año tendrás {edad + 1} años")
```

Trabajar con archivos

Python ofrece funciones simples para leer y escribir archivos, operaciones fundamentales en muchos programas.

```
# Escribir en un archivo
with open('datos.txt', 'w') as archivo:
    archivo.write("Hola desde Python\n")
    archivo.write("Esta es la segunda línea")

# Leer un archivo completo
with open('datos.txt', 'r') as archivo:
    contenido = archivo.read()
    print(contenido)
```

Formateo de cadenas

Python ofrece varias formas de formatear texto para presentar información de manera legible y organizada.

```
# Método format()
nombre = "María"
edad = 28
print("Hola, me llamo {} y tengo {} años.".format(nombre, edad))

# f-strings (Python 3.6+)
print(f"Hola, me llamo {nombre} y tengo {edad} años.")

# Formateo de números
precio = 19.99
print(f"El precio es: {precio:.2f} euros")
```

Buenas prácticas para I/O

- Siempre usa el bloque `with` al trabajar con archivos para asegurar que se cierren correctamente
- Verifica y valida las entradas del usuario antes de procesarlas
- Utiliza f-strings para formatear texto (más legible y eficiente)
- Maneja posibles errores al abrir, leer o escribir archivos con bloques `try-except`

Funciones I/O adicionales

Python ofrece muchas más funciones y módulos para operaciones de I/O avanzadas:

`json`: Para trabajar con datos JSON

`csv`: Para manejar archivos CSV

`pickle`: Para serializar objetos Python

`os`: Para operaciones del sistema de archivos

Buenas Prácticas en Python

PEP 8: Guía de Estilo

PEP 8 es la guía de estilo oficial para Python. Sus recomendaciones incluyen:

- Usar 4 espacios para indentación (no tabuladores)
- Limitar las líneas a 79 caracteres
- Usar líneas en blanco para separar funciones y clases
- Usar espacios alrededor de operadores
- Seguir convenciones de nombrado:
 - snake_case para variables y funciones
 - CamelCase para clases

Código Limpio y Legible

❌ **Mal estilo:**

```
def f(x,y):  
    z=x+y  
    return z
```

✅ **Buen estilo:**

```
def suma(x, y):  
    resultado = x + y  
    return resultado
```

Un código limpio es más fácil de entender, mantener y depurar, tanto para ti como para otros programadores.

Documentación y Comentarios

Documenta tus funciones y módulos usando docstrings:

```
def calcular_area_rectangulo(base, altura):  
    """  
    Calcula el área de un rectángulo.  
  
    Args:  
        base (float): La base del rectángulo  
        altura (float): La altura del rectángulo  
  
    Returns:  
        float: El área del rectángulo  
    """  
    return base * altura
```

Principios de Programación

- **DRY** (Don't Repeat Yourself): Evita duplicar código creando funciones reutilizables
- **KISS** (Keep It Simple, Stupid): Busca la solución más simple y directa
- **Modularidad**: Divide tu código en componentes pequeños y manejables
- **Consistencia**: Mantén un estilo coherente en todo tu código

Herramientas útiles

Estas herramientas te ayudarán a seguir buenas prácticas automáticamente:

pylint: Analizador de código

black: Formateador de código

flake8: Verificador de estilo

isort: Ordena importaciones

pytest: Marco de pruebas

mypy: Verificador de tipos

Recuerda el Zen de Python

*"Hermoso es mejor que feo.
Explícito es mejor que implícito.
Simple es mejor que complejo.
La legibilidad cuenta."*